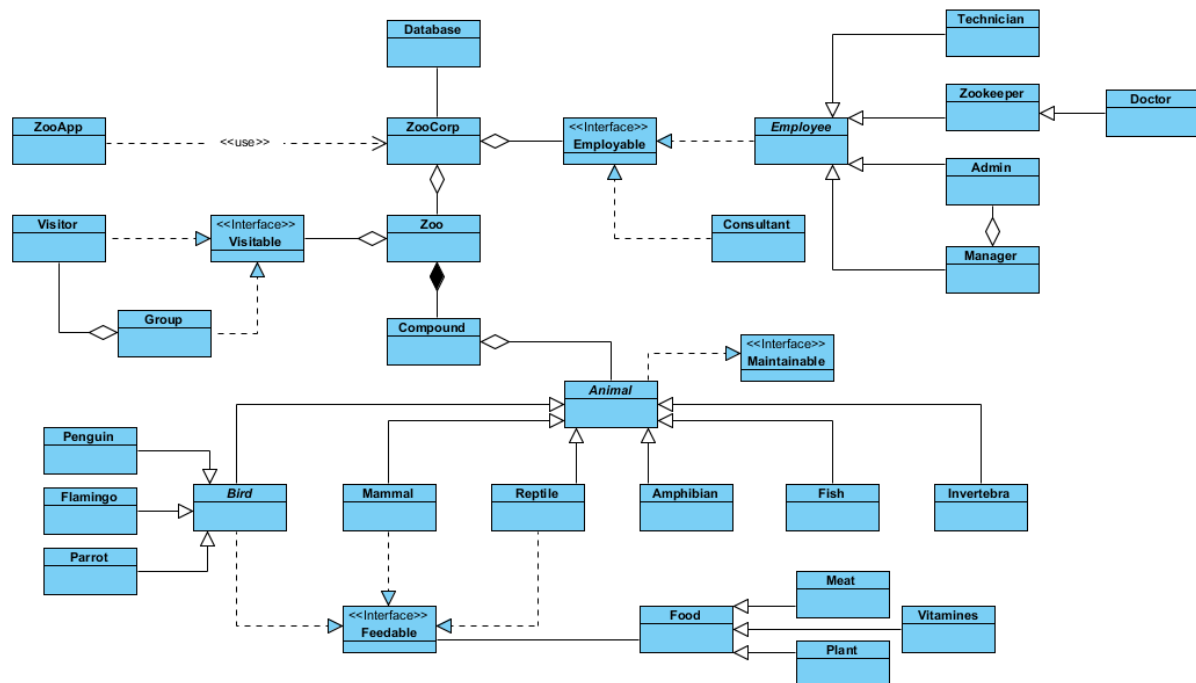


IDLAB 1: BUILDING THE ZOO EXAMPLE

Aims:

- Install IntelliJ IDEA Community Edition (scroll down the website)
 - <https://www.jetbrains.com/idea/download/#section=mac> – this is for MAC
 - <https://www.jetbrains.com/idea/download/#section=windows> – this for Windows
 - <https://www.jetbrains.com/idea/download/#section=linux> – this for Linux
- Familiarise yourself with IntelliJ IDEA with this Zoo Application
- Refresh object-oriented programming through building an application
- Gain more experience with object-oriented programming
 - Remember that familiarity with this will not only help you write better code, but will aid understanding of existing projects

Application



Coverage:

- In Part 1 (first half of this worksheet) we cover:
 - setters, getters, static variables, ArrayLists, and IDE shortcuts
- In Part 2 (second half of this worksheet) we cover:
 - Aggregation vs Composition
 - Writing the Compound, Animal and Employee classes for the Zoo example (includes an abstract class and an interface)

Doing everything from scratch may go beyond the lab session time, so we encourage you to start proactively and use the lab time for support.

Please create a repository of Lab1 in <https://projects.cs.nott.ac.uk/>.

AI Usage: We would like to see how AI used by students in a programming project. Please add link(s) in the README.md

Consent: Please complete the consent form here on AI Usage: <https://moodle.nottingham.ac.uk/mod/choice/view.php?id=9339127>

PART 1

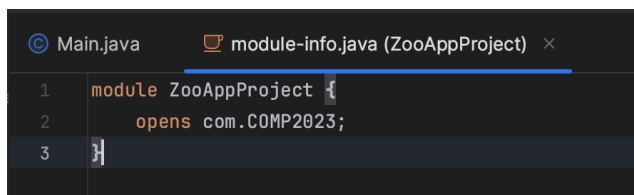
THE ZOO CLASS

-> First, we need to write the Zoo class, and to do that, we need to create a new project in IntelliJ.

-> First create a new package and call it whatever you like (e.g. "com.COMP2013").

-> Add a module-info.java file by right clicking on the [src] folder in the Project window and choosing {New > module-info.java} from the appearing context menu.

-> Then, in the module-info.java file type "opens" and the name you have chosen as your package name (i.e. like you did in the previous lab).



```
1 module ZooAppProject {
2     opens com.COMP2013;
3 }
```

-> Inside the package create a new Zoo class and add the following code. (Remember that you can also use IntelliJ to generate getters/setter for you if you want: add the private "location" field of type String, hover over the field, click it, and chose "More Actions..." from the appearing context menu.

-> Then chose the action to generate getters and setters. You can also auto-generate constructors. An alternative approach is to use {Code > Generate ...} from the menu bar.

```

1 package com.COMP2023;
2
3 public class Zoo {
4     private String location;
5
6     public Zoo(String location) {
7         this.setLocation(location);
8     }
9     public Zoo() {
10        this.setLocation("unknown");
11    }
12    public String getLocation() {
13        return location;
14    }
15    public void setLocation(String location) {
16        this.location = location;
17    }
18
19 }
20

```

- Add a private variable to store the number of compounds the zoo has.
- Add a line to the default constructor to set this to a default value (say, 30 compounds). Add a parameter to the other constructor to read in any value for this.
- Add a method called "buildNewCompound()" which adds one to this variable.

Now, so far we have just designed the class. Let's now actually create a real instance of it in our program (the object itself).

- Add a "ZooApp" class and "main" method, as before.
- Create two new "Zoo" objects. For one, use the default constructor (ie. don't pass any parameters on construction). For the second, pass an exotic location of your choice, and number of compounds.

Let's now add a method, which prints to the screen information about the zoo. Rather unexcitingly, let's call it "printInfo".

- Add the "printInfo" method to the Zoo class. It should return a String with the zoo's location and number of compounds. *Should this be public or private?*
- Add code to the ZooApp class to make use of it for both zoos. Then do a test run.

```

1  package com.COMP2023;
2
3  public class ZooApp {
4      public static void main(String[] args) {
5
6          System.out.println("Hello world!");
7          Zoo zoo1 = new Zoo();
8          Zoo zoo2 = new Zoo( location: "London", numCompound: 43);
9          System.out.println(zoo1.printInfo());
10         System.out.println(zoo2.printInfo());
11     }
12 }
13

```

Maybe pause here and have a look at some useful **shortcuts** in IntelliJ. They will help you to write code a lot faster, and these labs are the perfect place to try them out. You can find some suggestions for which are the best online.

THE CONCEPT OF A COMPANY OWNING LOTS OF ZOOS

As you saw in the previous step, we can have multiple version of our zoo classes. Why would we want to do this? Suppose you are a company who owns five zoos around the world, in London, Tokyo, New York, Paris, and Beeston.

- In your main code, create five zoo objects to represent this. You can set the compound numbers as you wish.
- Print out the info for all five zoos.

Suppose you want to assign each Zoo an individual Zoo ID number.

- Think about how you could do this **WITHIN** the objects themselves
- We want the first zoo created to have ID 1, second zoo ID 2 etc.
- When you have thought of an idea, then follow the example below...

You can do this using the static variable type. These variables are shared and accessible between all objects of the same type. First, we need to count how many zoos we have. So, we can share our zoo count (called "numZoo" below) between all the objects.

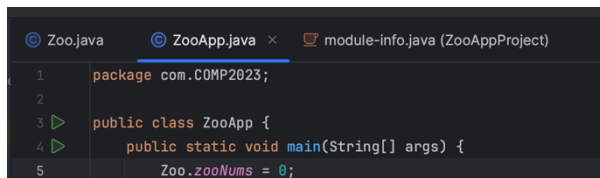
Add a static int variable called "numZoo":

```

3  public class Zoo {
4      private String location;
5      private int numCompound;
6      public static int numZoo;
7

```

You can actually access static variables *without* first creating an object. They don't need an object as they are really related to the class itself - all objects of the same class share the variable. Look - I can set its value to 0 **without** having to create an object first!



```

1 package com.COMP2023;
2
3 public class ZooApp {
4     public static void main(String[] args) {
5         Zoo.zooNums = 0;

```

(The variable is public so you can do this. If it were private, you could not access it outside the class like this, and would have to use a setter method.)

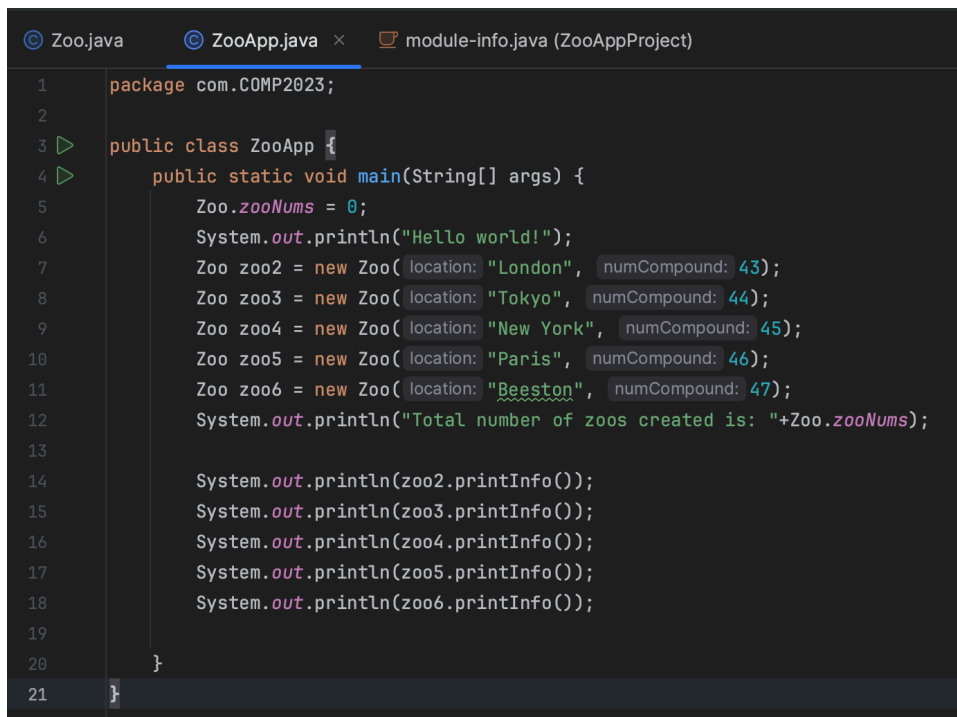
Remember, static means the variable is shared/accessible between all object instances. Let's see what that means if we want to use it to count how many zoos there are.

- Every time we create a new zoo, let's add one to this shared variable.

Task: Zoo ID Increment

- Work out where in the zoo class you need to handle this increment of our new variable, and add code to do it.

Now print out the number of zoos after the five we have created:

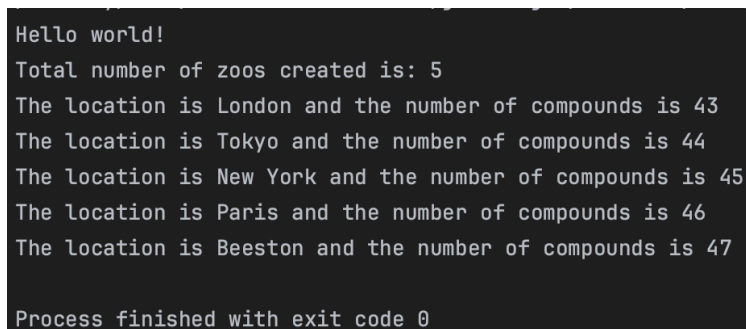


```

1 package com.COMP2023;
2
3 public class ZooApp {
4     public static void main(String[] args) {
5         Zoo.zooNums = 0;
6         System.out.println("Hello world!");
7         Zoo zoo2 = new Zoo( location: "London", numCompound: 43);
8         Zoo zoo3 = new Zoo( location: "Tokyo", numCompound: 44);
9         Zoo zoo4 = new Zoo( location: "New York", numCompound: 45);
10        Zoo zoo5 = new Zoo( location: "Paris", numCompound: 46);
11        Zoo zoo6 = new Zoo( location: "Beeston", numCompound: 47);
12        System.out.println("Total number of zoos created is: "+Zoo.zooNums);
13
14        System.out.println(zoo2.printInfo());
15        System.out.println(zoo3.printInfo());
16        System.out.println(zoo4.printInfo());
17        System.out.println(zoo5.printInfo());
18        System.out.println(zoo6.printInfo());
19    }
20 }
21

```

Your output should be:



```

Hello world!
Total number of zoos created is: 5
The location is London and the number of compounds is 43
The location is Tokyo and the number of compounds is 44
The location is New York and the number of compounds is 45
The location is Paris and the number of compounds is 46
The location is Beeston and the number of compounds is 47

Process finished with exit code 0

```

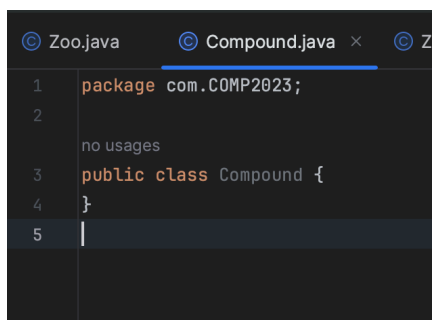
- You can use this now to set an individual unique ID number ("private int zooid") per Zoo in the constructor ("zooid=numZoos;"). For the "zooid" you should only create a getter but not a setter (or create one but set it to private). **Why is this the case?**

So static variables are an easy way to share data between objects of the same type. Make sure you are happy with this idea – it's an important concept. Please ask a lab helper if you would like more explanation.

ADDING ACTUAL COMPOUNDS WE COULD PUT ANIMALS IN: USING COLLECTIONS

So, we have the concept of a zoo, but at the moment it's not very interesting. We need a new class to represent our compounds (or enclosures), such as tanks, cages, fields etc.

- Add a new Compound class:



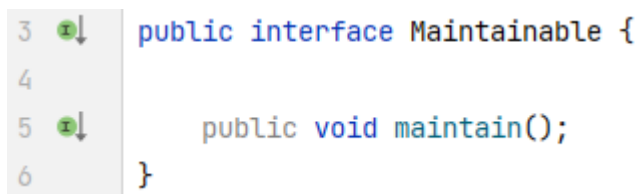
```

1 package com.COMP2023;
2
3 no usages
4 public class Compound {
5 }

```

One zoo can have many compounds. Complete the Compound class. Refer to the class diagram if needed.

- Create an interface called Maintainable

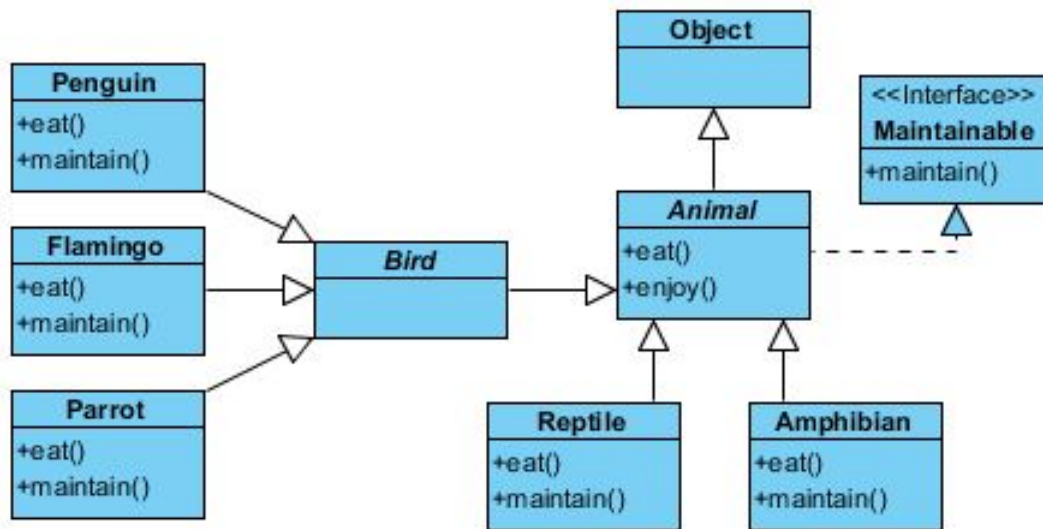


```

3 public interface Maintainable {
4
5     public void maintain();
6 }

```

- Now create an abstract class that implements Maintainable interface.



```

3 public abstract class Animal implements Maintainable {
4     private String name;
5
6     public Animal(String name) { this.setName(name); }
7
8
9     public abstract void eat();
10
11
12     public void enjoy() { System.out.println(this.getClass().getSimpleName()+" enjoys life as animal."); }
13
14
15     public String getName() { return name; }
16
17
18     public void setName(String name) { this.name = name; }
19
20
21 }
22
23

```

- Now use your imagination to create different animals. Remember to override maintain method from the Maintainable interface with appropriate maintenance activity. This is polymorphism in action!!

We can use a *Collection*

- Java has built-in data structures for handling lists etc.

We will use an "ArrayList". Add this to the list of variables in the Zoo class:

```

12 usages
5 public class Zoo {
6     2 usages
7     private String location;
8     4 usages
9     private int numCompound;
10    6 usages
11    public static int zooNums;
12    3 usages
13    private int zooId;
14    no usages
15    private ArrayList<Compound> compounds;
16
17 }

```

```

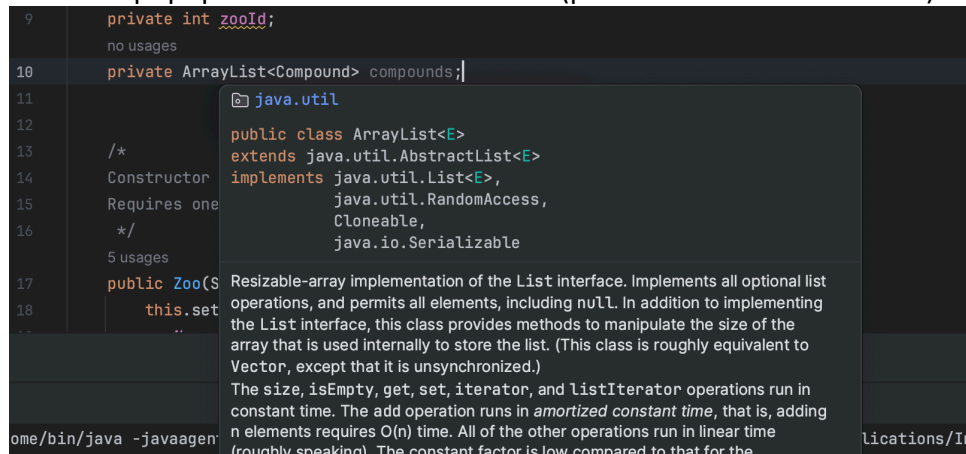
1 package com.COMP2023;
2
3 import java.util.ArrayList;

```

Note that if IntelliJ is not happy is because it doesn't know what an ArrayList is, and therefore printed it red. Hoover over it, and choose "Import Class" from the appearing context menu. This will automatically add the required import statement to tell Java how to find the relevant library. Sometimes this is automatic.

An ArrayList is really just an array, but suitable for storing objects, and with a number of helper methods. It is dynamic – that is, it can grow as the number of elements grows.

- To see more about what an ArrayList is, position the mouse inside the word ArrayList, and wait. A popup with detailed information (parts of the related Javadoc) will appear:



- Now add the highlighted text below to the Zoo constructor. This will create all our Compound objects for us:

```
public Zoo(String location, int numCompound) {
    this.setLocation(location);
    this.setCompounds(numCompound);

    zooNums++;
    this.setZooId(zooNums);
    for (int i=0; i<numCompound; i++){
        addCompound(new Compound());
    }
}
```

- Note how we can create **new** objects inline like this. Also note, we haven't written an addCompound() method yet! Don't panic. IntelliJ can help! Hoover over the red "addCompound" and in the appearing popup choose "Create method 'addCompound' in 'Zoo'"



As if by magic, the following method should be produced. Note IntelliJ has figured out the parameter type it needs.

- This auto-generation is useful for Test Driven Development, which we will introduce later, as you can write the test before you have written the code, and generate the basic code outline from the test code itself.

Now you just need to add the actual code inside the method:

```
1 usage
private void addCompound(Compound compound) {
    this.compounds.add(compound);
}
```

If you run this, you will get an error, as we haven't created our **compounds** object yet.

- Add this line to the constructor. It instantiates our ArrayList object and gets it ready to accept objects of type Compound.

```
5 usages
public Zoo(String location, int numCompound) {
    this.setLocation(location);
    this.setCompounds(numCompound);
    compounds = new ArrayList<Compound>();
    zooNums++;
    this.setZooId(zooNums);
    for (int i=0; i<numCompound; i++){
        addCompound(new Compound());
    }
}
```

Note that I am using set methods wherever possible.

- Now the code should compile! You are creating a number of new "Compound" objects, equal to the "numCompounds" variable.

Task: Extend the Zoo project

- Using the same idea, change the "Compounds" class so you can add animals, by using an ArrayList again. You will need to create a class Animal and you will need a method to add animal objects to your new ArrayList.
- Note: We will look at how we can use abstract classes and inheritance with animals in Part 2 of this lab.

ADDING SOME EMPLOYEES

The diagram illustrates a Zoo Management System architecture. It features several classes and interfaces with their relationships:

- Database** is associated with **ZooCorp**.
- ZooCorp** is associated with **Zoo** and **Employee** (via **Employable** interface).
- Zoo** is associated with **Compound** and **Visitor** (via **Visitable** interface).
- Visitor** is associated with **Group**.
- Compound** is associated with **Animal** and **Maintainable** interface.
- Animal** is a base class for **Bird**, **Mammal**, **Reptile**, **Amphibian**, **Fish**, and **Invertebra**.
- Bird** is associated with **Penguin**, **Flamingo**, and **Parrot**.
- Mammal** is associated with **Feeder** interface.
- Reptile** is associated with **Feeder** interface.
- Feeder** interface is associated with **Food** and **Plant**.
- Food** is associated with **Meat** and **Vitamines**.
- Plant** is associated with **Vitamines**.
- Employee** is a base class for **Technician**, **Zookeeper**, **Admin**, **Manager**, and **Consultant**.
- Doctor** is associated with **Zookeeper**.

ZooCorp:

So, let's allow Zoo objects to exist outside of the ZooCorp class. It makes sense to have a ZooCorp constructor, which accepts a Zoo object, so let's add that as an option. A good data structure to use might be the ArrayList again, as we used in Part 1 e.g.

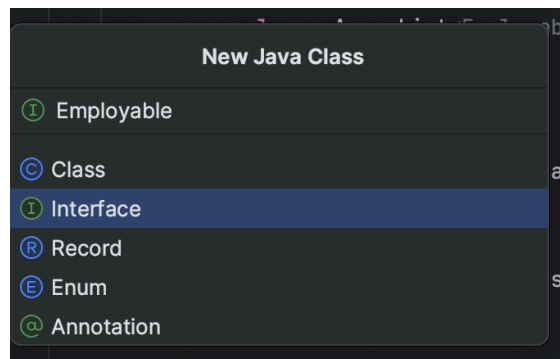
```
6 private ArrayList<Zoo> zoos;
```

- Create a class ZooCorp
- Write a constructor for ZooCorp objects, which takes a Zoo object as an argument and stores it in a newly created ArrayList.
- Then, write a method called addZoo(), which also takes a Zoo object as an argument and adds it to the ArrayList.

WRITING THE EMPLOYEE CLASSES

Look again at the class diagram above. To write the employee classes, we need to write an interface, as they are written using one called *Employable*. We will think about the details of this in a while, but for now let's just create one:

- In the Project window right click on "com.COMP2013" and choose {New > Java Class > Interface}



If you remember, an interface is simply a blueprint for your class: it lists the method names that the class *must* implement. This ensures that any class, which uses that interface, has at least those methods accessible publicly, for other parts of the program to use.

Why use interfaces? Think of it like this. Whatever TV you have at home - you can be sure it has an on/off button. So, to represent this, in programming you could write an interface called *Switchable* which has two methods, *switchOn()* and *switchOff()*. Then *any* brand of TV can implement this interface, and you know you can turn it on and off, using the methods it provides.

Here, we are interested in objects, which are *employable* – as they represent staff at ZooCrop. All *Employable* objects should implement the following methods, so add them to the interface:

```

1  package com.COMP2013;
2
3  public interface Employable {
4      public void setEmployeeID(int number);
5      public int getEmployeeID();
6      public void setEmployeeName(String name);
7      public String getEmployeeName();
8      public int getSalary();
9      public void setSalary(int salary);
10     public int calculateChristmasBonus();
11 }
12

```

Remember in an interface you just list the method signatures, NOT any code.

It is the job of the class *implementing* the interface to provide the code. We don't care *how* the method works at this stage, just what goes into it (parameters), and what returns from it.

To create a class which implements the interface:

Create a new class Employee and then **set the class to abstract**. Then add "implements Employable" after the class name.

```
public abstract class Employee implements Employable {
```

Note you can use interfaces in non-abstract classes just fine. However, let's do it here to match our diagram above, and to see what using an abstract class can add

Hover over Employable, right click on it and choose the option {Generate...} in the appearing context menu. Next choose {Implement Methods...} from the appearing popup and select all methods for implementation. Click {OK}.

So, Employee is an **abstract** class. This means we can't create an object directly from this class. This wouldn't make much sense – you can't have a generic Employee, everyone has a particular job (zookeeper, admin, etc.). However, an abstract class is more than just an interface – we can define some generic behaviour here, which all employee objects can use. Provide implementations for all the methods we have so far, as they are all very general in nature, in this abstract class:

Task

- Write the code for the stubs. You will need to add some variables.
- Note, **interfaces** do not define the required variables; this is an *implementation* choice we can now choose in the abstract class.

Now let's add a Zookeeper class, which inherits from this abstract Employee class: Then add "extends Employee" after the class name. This will now compile, as the Zookeeper has access to all the methods we have coded in the abstract class, Employee.

```
3 public class Zookeeper extends Employee {
4
1 usage
```

Let's show why you can't code everything in an abstract class. Now add another method to the Employable interface, called "calculateChristmasBonus".

- Add: public int calculateChristmasBonus() to the Employable interface

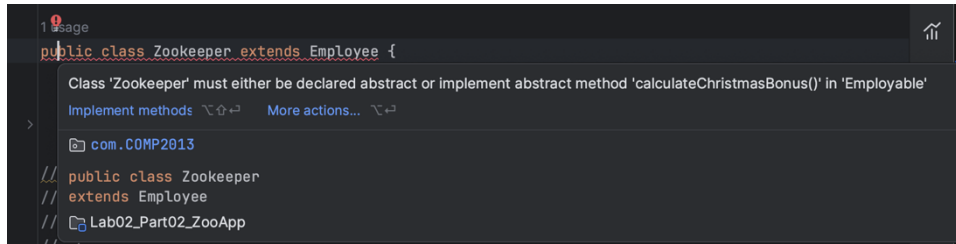
```
public int calculateChristmasBonus();
```

When you implement this, different roles in the Zoo have different bonuses:

- For a Zookeeper, the Christmas bonus equation is: $salary \times 0.05 + 100$
- For an Admin role, the bonus is: $salary \times 0.08$

So, you can't add the implementation into the generic abstract class. The code will no longer compile, as you need an implementation of this (because the interface says you need one!)

- Go to your Zookeeper class. Click the red underlined name of the class, and here IntelliJ can help us! Click "Implement methods" in the appearing popup.



Hey presto! The required method stub appears:

```

public class Zookeeper extends Employee {
    2 usages
    @Override
    public int calculateChristmasBonus() {
        return 0;
    }
  
```

- For your Zookeeper class, add the bonus equation ($salary \times 0.05 + 100$)
- Now add an Admin class, which also inherits from Employee, and implement the different Admin bonus equation ($salary \times 0.08$)

So, the **interface** says there HAS to be a method called calculateChristmasBonus which returns an integer. However, we can't put a generic calculation in the **abstract class** Employee because the exact calculation changes depending on the Employee type. So, we put the final implementation of this in our actual person classes (Zookeeper, Admin etc.).

Tasks: Tidy up...

- To tidy up, add a constructor for Zookeeper and Admin, to set a name on creation.
 - Could you put this constructor in the abstract Employee class instead? How?
- Modify ZooCorp, as you did for Zoo, to accept Employable people, and maintain an ArrayList of employees.
- Write code in the main to test everything. Below is an example, how this might look like.

```

5 public static void main(String[] args) {
6     Zoo.numZoo=0;
7     Zoo zoo1=new Zoo( location: "London", numCompounds: 20);
8     ZooCorp megaZoo=new ZooCorp(zoo1);
9     Zoo zoo2=new Zoo( location: "Tokyo", numCompounds: 15);
10    megaZoo.addZoo(zoo2);
11    Zoo zoo3=new Zoo( location: "New York", numCompounds: 26);
12    Zoo zoo4=new Zoo( location: "Paris", numCompounds: 18);
13    Zoo zoo5=new Zoo( location: "Beeston", numCompounds: 5);
14    System.out.println(zoo1.printInfo());
15    System.out.println(zoo2.printInfo());
16    System.out.println(zoo3.printInfo());
17    System.out.println(zoo4.printInfo());
18    System.out.println(zoo5.printInfo());
19    zoo5.buildNewCompound();
20    System.out.println(zoo5.printInfo());
21    System.out.println("Number of Zoos:"+Zoo.numZoo);
22    Employee sue=new Zookeeper( name: "Sue");
23    Employee bob=new Admin( name: "Bob");
24    megaZoo.addStaff(sue);
25    megaZoo.addStaff(bob);
26    System.out.println(sue.getClass().getSimpleName()+" "+sue.getEmployeeName()+" "+sue.calculateChristmasBonus());
27    System.out.println(bob.getClass().getSimpleName()+" "+bob.getEmployeeName()+" "+bob.calculateChristmasBonus());
28 }
29
30
  
```

Challenges: Think about some suitable use case and add to the ZooApp class?

That's all for this labsheet. We hope you enjoyed it!

Lab 01 - Zoo OOP Core Revision Summary

Java OOP refresh · classes/objects · getters/setters · static · ArrayList · aggregation/composition · inheritance · abstract classes · interfaces · polymorphism · Git workflow

1. Topic Overview

This lab rebuilds the core Java object-oriented model through a small zoo application. The progression is deliberate: start with a single Zoo class, then add shared class state, collections of objects, ownership relationships, abstract base classes, interfaces, inheritance and polymorphic behaviour. The completed implementation links these ideas into one object graph rather than treating them as isolated syntax.

Lab logic: Zoo object → getters/setters + constructors → static zoo counter / per-object ID → ArrayList<Compound> → Compound stores Animals → abstract/interface hierarchy → ZooCorp aggregation → Employable/Employee polymorphism.

2. Core Concepts

Classes, objects, constructors and encapsulation

- ▶ Class = blueprint; object = concrete instance created with new. Multiple Zoo objects share class structure but keep separate instance state.
- ▶ Constructors establish valid initial state. The lab uses a default Zoo constructor and a parameterised Zoo(String location, int numCompounds).
- ▶ Getters/setters expose controlled access to private fields. A generated ID should have a getter but no public setter because callers should not arbitrarily rewrite object identity.

Instance state vs static state

- ▶ zooID is an instance field: every Zoo has its own value.
- ▶ numZooos is static: one value is shared by all Zoo instances and belongs conceptually to the class.

Creation rule: numZooos++; then this.zooID = numZooos; → first Zoo gets ID 1, second gets ID 2, etc.

Collections and real object state

- ▶ ArrayList<Compound> stores the actual Compound objects; numCompounds is only a count. Keeping both means they must stay synchronised.
- ▶ Declaring private ArrayList<Compound> compounds; does not create a list. The constructor must execute compounds = new ArrayList<>(); before calling add().
- ▶ buildNewCompound() should both add a new Compound object and increment the count; otherwise logical state and collection state diverge.
- ▶ Compound repeats the same pattern using ArrayList<Animal> and addAnimal(Animal).

3. Key Mechanisms / Implementation Patterns

1) Zoo construction and contained objects

```
compounds = new ArrayList<>();
numZoos++;
this.zooID = numZoos;
for (int i = 0; i < numCompounds; i++)
    addCompound(new Compound());
```

The constructor creates the collection first, assigns the class-wide counter-derived ID, then materialises the requested number of Compound objects.

2) Abstract class + interface: Animal

Maintainable specifies `maintain()`. Animal implements the interface, stores shared state such as name, provides common behaviour such as `enjoy()`, and leaves `eat()` abstract.

- ▶ A concrete animal must supply the missing behaviour. Penguin and Parrot implement their own `eat()` and `maintain()`.
- ▶ Strict UML hierarchy shown in the worksheet: Animal → Bird → Penguin / Flamingo / Parrot, plus direct Animal subclasses Reptile and Amphibian. Bird is shown as an abstract intermediate class.

3) Polymorphism through an abstract reference

```
Animal penguin = new Penguin("Pip");
Animal parrot = new Parrot("Rio");
for (Animal animal : new Animal[] {penguin, parrot}) {
    animal.eat();
    animal.maintain();
    animal.enjoy();
}
```

The variable type is Animal, but Java dispatches overridden methods according to the runtime object type. The same call therefore produces different behaviour.

4) Aggregation and composition in the Zoo model

- ▶ ZooCorp ◇ Zoo (aggregation): a Zoo can exist independently and later be added to a ZooCorp. The constructor receives an already-created Zoo; later Zoos use `addZoo()`.
- ▶ Zoo ◆ Compound (composition in the UML): the Zoo constructor creates its Compound objects; their lifecycle is modelled as strongly owned by the Zoo.
- ▶ Compound ◇ Animal (aggregation in the UML): Animal objects can be created independently and then inserted into a Compound using `addAnimal()`.

5) Employable → Employee → concrete roles

- ▶ Employable interface defines the public contract: employee ID/name access, salary access and `calculateChristmasBonus()`.
- ▶ Employee is abstract: it implements generic storage/getter/setter behaviour but represents no specific job role.
- ▶ Zookeeper and Admin inherit Employee and supply role-specific bonus calculations.

Bonus rules: Zookeeper = salary × 0.05 + 100 | Admin = salary × 0.08

6) Constructor chaining with super

```
public Employee(String name) { this.employeeName = name; }
public Zookeeper(String name) { super(name); }
public Admin(String name) { super(name); }
```

`super(name)` delegates common initialisation to the parent constructor instead of duplicating name-setting logic in every subclass.

4. Rules / Conditions / Design Constraints

- ▶ Initialise collections before use. Calling `add()` on an uninitialised `ArrayList` reference causes a `NullPointerException`.
- ▶ Keep duplicate representations consistent. If both `numCompounds` and `compounds.size()` exist, methods that resize one should update the other.
- ▶ Abstract classes cannot be instantiated. `new Animal(...)` and `new Employee(...)` are invalid; instantiate concrete subclasses.
- ▶ Interfaces specify capability, not storage. `Employable` lists method signatures; `Employee` chooses the fields needed to implement them.
- ▶ Concrete subclasses must satisfy inherited abstract/interface obligations. If `Employee` does not implement `calculateChristmasBonus()`, each concrete role must.
- ▶ Prefer the abstraction in collections. `ArrayList<Employable>` allows any future class that implements `Employable` to be stored without changing `ZooCorp`.
- ▶ Numeric subtlety: bonus formulas produce a double; the lab implementation casts to `int`, so any fractional part is truncated.
- ▶ final collection reference: IntelliJ may suggest `final ArrayList<...>`. `final` prevents reassignment of the list reference; it does not prevent `add()/remove()`.

5. Comparison & Common Confusions

Concept A	Concept B	Key Difference / Revision Point
Instance field	static field	Instance field belongs to each object; static field is shared by all instances of the class.
Declaration	Instantiation	<code>ArrayList<T> x;</code> only declares a reference; <code>new ArrayList<>()</code> creates the object.
Interface	Abstract class	Interface defines a capability/contract; abstract class can also hold fields, constructors and shared implementation.
Inheritance	Composition / aggregation	Inheritance models “is-a”; containment models “has-a”. Use them for different relationships.
Aggregation	Composition	Aggregation allows independent lifetimes (<code>ZooCorp-Zoo</code>); composition models stronger lifecycle ownership (<code>Zoo-Compound</code>).
Compile-time reference type	Runtime object type	<code>Animal a = new Penguin(...)</code> : accessible API is constrained by <code>Animal</code> , overridden dispatch uses <code>Penguin</code> at runtime.
Overloading	Overriding	Overloading changes parameter lists; overriding replaces inherited behaviour with the same signature.
Getter only ID	Getter + setter property	Generated identity should not be externally mutable; ordinary mutable state may expose a setter.
Git warning	Java/compiler error	Yellow IntelliJ inspections can be committed; red compile errors block successful compilation/run.

6. Exam / Revision Focus

Must Know

- ▶ Definitions and purpose: class, object, constructor, getter/setter, static, ArrayList, interface, abstract class, inheritance, overriding, polymorphism, aggregation, composition.
- ▶ Zoo identity pattern: shared numZoos + per-instance zooID.
- ▶ Why ArrayList<Compound> / ArrayList<Animal> are used: dynamic collections of objects with common types.
- ▶ Animal design: Animal implements Maintainable; concrete animals override behaviour.
- ▶ Employee design: Employee implements Employable; role-specific bonus lives in concrete subclasses.

Must Understand

- ▶ Why a static variable can count objects while an instance field stores each object's own ID.
- ▶ Why a collection declaration must be followed by object construction before use.
- ▶ Why abstract classes centralise shared state/behaviour while interfaces define a reusable contract.
- ▶ How polymorphism lets the same method call execute different subclass implementations.
- ▶ Why ZooCorp-Zoo is aggregation: the Zoo exists first and is passed into / added to the company.
- ▶ Why super(name) is preferable to duplicating parent initialisation in every subclass.

Easy to Confuse

- ▶ static ≠ constant. A static field is shared; it can still change unless declared final.
- ▶ final ArrayList ≠ immutable list. The reference cannot be reassigned, but its contents can still change.
- ▶ Interface ≠ abstract class. In this lab, Employable/Maintainable define required behaviour; Employee/Animal provide shared implementation/state.
- ▶ Count ≠ collection contents. Changing only numCompounds does not create/remove Compound objects.
- ▶ Compile-time type ≠ runtime behaviour. An Animal reference can point at Penguin/Parrot; overridden methods dispatch dynamically.
- ▶ Warning ≠ failure. IntelliJ code-analysis warnings such as "collection updated but never queried" are inspections, not Java compile errors.

Practical workflow remembered from this lab

- ▶ JDK: JDK 21 LTS selected for a stable course environment; duplicate IntelliJ entries can be symlink paths rather than duplicate installations.
- ▶ Git: local project linked to the school GitLab remote; normal cycle is add → commit → push.
- ▶ GitLab SSO: HTTPS authentication uses a Personal Access Token instead of the school password. IntelliJ GitLab integration additionally needs API/user-reading scopes.
- ▶ README: labsheet requests AI-usage link(s); keep the repository private unless the course says otherwise.

Implementation checkpoint: the completed project currently includes Zoo, Compound, Maintainable, Animal, Penguin, Parrot, ZooCorp, Employable, Employee, Zookeeper and Admin. Strictly mirroring the worksheet's animal UML would additionally insert abstract Bird and add Flamingo, Reptile and Amphibian; the larger overview UML also shows Mammal, Fish, Invertebrate and feeding-related classes that were not required for the implemented core path.

Source emphasis incorporated: COMP2013 Lab 01 worksheet pp.2-13; completed project structure and debugging/implementation decisions from the lab walkthrough.